

BERT y variantes orientadas a modelos multimodales

Documento de ejemplos, conceptos y puente hacia VisualBERT, UNITER, ViLT, BLIP-2 y LLaVA

Material de apoyo para IA Generativa y Aprendizaje Multimodal

Índice

1. Propósito del documento	2
2. BERT como punto de partida	2
2.1. Arquitectura general	2
2.2. Pretraining y fine-tuning	2
3. Entrada de BERT: tokens, segmentos y posiciones	2
3.1. Ejemplo manual	2
3.2. Código didáctico	3
4. Atención bidireccional y ventana de contexto	3
4.1. Qué mira la atención	3
4.2. Código de máscaras	4
5. Masked Language Modeling y el valor labels = -100	4
5.1. Idea del objetivo	4
5.2. Código didáctico	4
6. Regularización en BERT	5
6.1. Dropout	5
6.2. Código didáctico	5
7. De BERT a VisualBERT	6
7.1. Secuencia multimodal	6
7.2. Ejemplo manual	6
7.3. Código didáctico	6
8. UNITER y alineamiento palabra-región	7
8.1. Qué agrega UNITER	7
8.2. Error de grounding	7
9. ViLT y patches visuales	7
9.1. Eliminar el detector pesado	7
9.2. Ejemplo manual	8
10. BLIP-2 y Q-Former	8
10.1. Problema que resuelve	8
10.2. Ejemplo manual	8
10.3. Código didáctico	8
11. LLaVA y proyección visual hacia un LLM	9
11.1. Idea central	9
11.2. Código didáctico	9
11.3. Ejemplo manual	9
12. Comparación técnica de variantes	9
13. Ejemplos integradores	10
13.1. Ejemplo 1: ventana de contexto multimodal	10
13.2. Ejemplo 2: clasificación con [CLS]	10
13.3. Ejemplo 3: error de alineamiento	10

14. Evaluación y criterios de calidad	11
14.1. Qué debe medirse	11
14.2. Taxonomía mínima de errores	11
15. Ruta conceptual final	12
16. Glosario breve	12
17. Conclusión	12

1. Propósito del documento

Este documento explica BERT y sus variantes orientadas a modelos multimodales con un enfoque técnico, gradual y operativo. El objetivo es que el estudiante entienda cómo se pasa de un Transformer textual de tipo encoder-only a arquitecturas que procesan texto, regiones visuales, patches, consultas aprendidas y embeddings proyectados hacia un modelo de lenguaje.

La idea principal es sencilla pero profunda: los modelos multimodales modernos convierten modalidades heterogéneas en secuencias de vectores compatibles con atención. BERT enseña la gramática básica: tokens, embeddings, posiciones, segmentos, máscaras, atención bidireccional, preentrenamiento y fine-tuning. VisualBERT, UNITER, ViLT, BLIP-2 y LLaVA reutilizan esa gramática, pero agregan evidencia visual y objetivos de alineamiento.

2. BERT como punto de partida

2.1. Arquitectura general

BERT es un Transformer de tipo encoder-only. Esto significa que no está diseñado primariamente para generar texto de izquierda a derecha, sino para construir representaciones contextuales de una secuencia completa. Su atención es bidireccional: cada token puede consultar tokens anteriores y posteriores dentro de la misma ventana de contexto.

Una entrada típica tiene esta forma:

```
[CLS] oracion A [SEP] oracion B [SEP]
```

El token [CLS] funciona como vector global de la secuencia. Se suele usar para tareas de clasificación. El token [SEP] separa segmentos. El token [MASK] aparece durante el preentrenamiento con Masked Language Modeling.

2.2. Pretraining y fine-tuning

Durante el pretraining, BERT aprende de grandes colecciones de texto no etiquetado. Su objetivo central es reconstruir tokens ocultos usando contexto izquierdo y derecho. En el BERT original también se usó Next Sentence Prediction, donde el modelo decide si una oración B continúa una oración A.

Durante el fine-tuning, se agrega una cabecera específica para la tarea. Por ejemplo:

- Clasificación: se usa [CLS] y una cabecera lineal.
- NER: se predice una etiqueta por token.
- SQuAD: se predicen posiciones de inicio y fin de una respuesta dentro de un párrafo.

3. Entrada de BERT: tokens, segmentos y posiciones

3.1. Ejemplo manual

Supongamos dos segmentos:

```
Texto A: my dog is cute  
Texto B: he likes playing
```

La entrada se organiza así:

```
[CLS] my dog is cute [SEP] he likes play ##ing [SEP]
```

La representación final de cada posición es una suma:

$$\mathbf{e}_i = \mathbf{e}_i^{token} + \mathbf{e}_i^{segmento} + \mathbf{e}_i^{posicion}.$$

Un ejemplo tabular sería:

Posición	Token	Segmento	Descripción
0	[CLS]	A	representación global
1	my	A	primer segmento
2	dog	A	primer segmento
3	is	A	primer segmento
4	cute	A	primer segmento
5	[SEP]	A	separador
6	he	B	segundo segmento
7	likes	B	segundo segmento
8	play	B	segundo segmento
9	##ing	B	subpalabra WordPiece
10	[SEP]	B	cierre

3.2. Código didáctico

El siguiente código reproduce la lógica básica de construcción de entrada. La firma de función está en inglés, pero los comentarios y cadenas están en español.

```
def build_bert_input(tokens_a, tokens_b=None, max_len=16):
    """Construye una entrada tipo BERT con ids de segmento y mascara."""
    tokens = ["[CLS]"] + tokens_a + ["[SEP]"]
    token_type_ids = [0] * len(tokens)

    if tokens_b is not None:
        tokens = tokens + tokens_b + ["[SEP]"]
        token_type_ids = token_type_ids + [1] * (len(tokens_b) + 1)

    tokens = tokens[:max_len]
    token_type_ids = token_type_ids[:max_len]
    attention_mask = [1] * len(tokens)

    while len(tokens) < max_len:
        tokens.append("[PAD]")
        token_type_ids.append(0)
        attention_mask.append(0)

    return tokens, token_type_ids, attention_mask
```

4. Atención bidireccional y ventana de contexto

4.1. Qué mira la atención

Si una secuencia tiene 50 tokens válidos, una capa Transformer calcula una matriz de atención de tamaño 50×50 por cada head de atención. La fila i indica a qué posiciones atiende el token i . Matemáticamente, el token puede mirar toda la ventana válida. Semánticamente, la distribución suele concentrar mayor peso en algunos tokens relevantes.

En BERT, la máscara es bidireccional:

	[CLS]	el	gato	mira	[SEP]
[CLS]	1	1	1	1	1
el	1	1	1	1	1
gato	1	1	1	1	1
mira	1	1	1	1	1
[SEP]	1	1	1	1	1

En un decoder causal, la máscara sería triangular:

	[CLS]	el	gato	mira	[SEP]
[CLS]	1	0	0	0	0
el	1	1	0	0	0
gato	1	1	1	0	0
mira	1	1	1	1	0
[SEP]	1	1	1	1	1

4.2. Código de máscaras

```
import torch

def make_bidirectional_mask(num_tokens):
    """Crea una mascara bidireccional para tokens validos."""
    return torch.ones(num_tokens, num_tokens, dtype=torch.int64)

def make_causal_mask(num_tokens):
    """Crea una mascara causal para generacion autoregresiva."""
    return torch.tril(torch.ones(num_tokens, num_tokens, dtype=torch.int64))
```

5. Masked Language Modeling y el valor labels = -100

5.1. Idea del objetivo

En Masked Language Modeling, BERT recibe una secuencia donde algunos tokens fueron reemplazados por [MASK]. La pérdida se calcula solo en esas posiciones. Para ignorar posiciones no enmascaradas, en PyTorch se usa con frecuencia el valor -100, porque CrossEntropyLoss lo puede tratar como ignore_index.

Ejemplo:

```
Texto original:
[CLS] el gato mira la luna [SEP]

Entrada al modelo:
[CLS] el [MASK] mira la luna [SEP]

labels:
-100 -100 gato -100 -100 -100 -100
```

El modelo produce logits para todas las posiciones, pero la pérdida solo se calcula en la posición donde la etiqueta no es -100.

5.2. Código didáctico

```
def apply_mlm_mask(input_ids, mask_id, special_ids, probability=0.15):
    """Aplica mascara MLM y usa -100 para ignorar tokens no evaluados."""
    labels = torch.full_like(input_ids, fill_value=-100)
    masked_input_ids = input_ids.clone()

    for position, token_id in enumerate(input_ids.tolist()):
        if token_id in special_ids:
            continue
        if torch.rand(1).item() < probability:
            labels[position] = token_id
            masked_input_ids[position] = mask_id

    return masked_input_ids, labels
```

6. Regularización en BERT

6.1. Dropout

BERT usa regularización. Una de las formas más importantes es dropout. Durante entrenamiento, dropout apaga aleatoriamente una fracción de activaciones. Durante inferencia, dropout se desactiva.

En un bloque Transformer aparece dropout en varios puntos:

- después de embeddings,
- sobre probabilidades de atención,
- después de feed-forward,
- antes de una cabecera de clasificación durante fine-tuning.

Un valor típico en BERT es 0.1 para `hidden_dropout_prob` y 0.1 para `attention_probs_dropout_prob`.

6.2. Código didáctico

```
import torch.nn as nn

class BertStyleBlock(nn.Module):
    """Bloque didactico con atencion, dropout y feed-forward."""

    def __init__(self, d_model=64, num_heads=4, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(
            embed_dim=d_model,
            num_heads=num_heads,
            dropout=dropout,
            batch_first=True,
        )
        self.dropout = nn.Dropout(dropout)
        self.norm_1 = nn.LayerNorm(d_model)
        self.norm_2 = nn.LayerNorm(d_model)
        self.feed_forward = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Dropout(dropout),
```

```

        nn.Linear(4 * d_model, d_model),
    )

    def forward(self, x, key_padding_mask=None):
        attn_output, _ = self.attention(
            x, x, x, key_padding_mask=key_padding_mask
        )
        x = self.norm_1(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm_2(x + self.dropout(ff_output))
        return x

```

7. De BERT a VisualBERT

7.1. Secuencia multimodal

VisualBERT extiende la intuición de BERT: en lugar de procesar solo tokens textuales, procesa tokens textuales y regiones visuales dentro de una misma secuencia. Si una imagen tiene regiones detectadas $R_1, R_2, \dots, R_N$, una entrada conceptual puede ser:

```
[CLS] que sostiene el niño [SEP] R1 R2 R3 ... RN
```

La ventana de contexto ahora contiene palabras y regiones. Si hay 7 tokens textuales, 43 regiones visuales y 0 padding, la matriz de atención efectiva es 50×50 por head y por capa.

7.2. Ejemplo manual

Imagen con regiones:

```

R1: niño
R2: pelota
R3: parque

```

Pregunta:

```
que sostiene el niño
```

La atención esperada podría concentrarse así:

Token textual	Región relevante	Interpretación
niño	R1	objeto principal
sostiene	R1 y R2	relación entre entidad y objeto
pelota implícita	R2	respuesta visual probable
[CLS]	R1, R2, R3	integración global

7.3. Código didáctico

```

def build_visualbert_sequence(text_embeddings, visual_embeddings):
    """Concatena tokens textuales y visuales en una secuencia compartida."""
    sequence = torch.cat([text_embeddings, visual_embeddings], dim=1)
    return sequence

batch_size = 2
text_tokens = 7
visual_tokens = 43

```



```
d_model = 64

text_embeddings = torch.randn(batch_size, text_tokens, d_model)
visual_embeddings = torch.randn(batch_size, visual_tokens, d_model)
sequence = build_visualbert_sequence(text_embeddings, visual_embeddings)

print("Forma de la secuencia multimodal:", sequence.shape)
```

8. UNITER y alineamiento palabra-región

8.1. Qué agrega UNITER

UNITER mantiene la lógica de fusión profunda, pero enfatiza objetivos multimodales para aprender alineamiento fino. No basta saber que una imagen y una frase coinciden globalmente. El modelo debe asociar palabras, objetos, atributos y relaciones con evidencia visual.

Ejemplo:

Texto: un niño monta una bicicleta en la calle
 Regiones: R1 = niño, R2 = bicicleta, R3 = calle

Alineamiento esperado:

Palabra	Región esperada	Tipo de evidencia
nino	R1	objeto
bicicleta	R2	objeto
monta	R1 y R2	acción y relación
calle	R3	contexto espacial

8.2. Error de grounding

Si el modelo atiende `bicicleta` -> R3, la oración puede sonar correcta, pero el grounding es débil. Esta distinción es central en evaluación multimodal: una respuesta correcta no basta si la evidencia visual no sostiene la respuesta.

9. ViLT y patches visuales

9.1. Eliminar el detector pesado

VisualBERT y UNITER suelen depender de regiones visuales producidas por un detector externo. ViLT simplifica el pipeline: usa patches de imagen como tokens visuales. Esto reduce dependencia de detectores, pero exige que el Transformer aprenda la fusión desde señales más crudas.

Ejemplo conceptual:

[CLS] el perro juega con una pelota [SEP] P1 P2 P3 P4 ... PN

Donde P_i son patches de la imagen.

9.2. Ejemplo manual

Patch	Contenido aproximado	Token textual asociado
P1	cielo	contexto
P2	arbol	contexto
P3	perro	perro
P4	pelota	pelota

En ViLT, el modelo debe aprender que **perro** se asocia con *P3* y **pelota** con *P4*, sin recibir una etiqueta explícita de detector.

10. BLIP-2 y Q-Former

10.1. Problema que resuelve

BLIP-2 conecta un encoder visual congelado con un modelo de lenguaje grande también congelado. Entrenar ambos de extremo a extremo sería costoso. Por eso introduce un módulo intermedio entrenable llamado Q-Former.

El Q-Former usa consultas aprendidas. Estas consultas no son palabras humanas. Son tokens entrenables que aprenden a extraer evidencia visual relevante desde los tokens del encoder visual.

10.2. Ejemplo manual

Supongamos que el encoder visual produce 196 tokens:

V1, V2, V3, ..., V196

El Q-Former usa pocas consultas:

Q1, Q2, Q3, Q4

Una interpretación didáctica puede ser:

Consulta	Evidencia extraída	Uso posterior
Q1	objeto principal	captioning
Q2	atributo visual	respuesta precisa
Q3	relación espacial	VQA
Q4	contexto de escena	diálogo visual

10.3. Código didáctico

```
class QFormerToy(nn.Module):
    """Q-Former didactico con consultas aprendidas y cross-attention."""

    def __init__(self, d_model=64, num_queries=8, num_heads=4):
        super().__init__()
        self.learned_queries = nn.Parameter(torch.randn(1, num_queries, d_model))
        self.cross_attention = nn.MultiheadAttention(
            embed_dim=d_model,
            num_heads=num_heads,
            batch_first=True,
        )

    def forward(self, visual_tokens):
        batch_size = visual_tokens.shape[0]
```

```

queries = self.learned_queries.expand(batch_size, -1, -1)
output, attention_weights = self.cross_attention(
    queries, visual_tokens, visual_tokens
)
return output, attention_weights

```

11. LLaVA y proyección visual hacia un LLM

11.1. Idea central

LLaVA toma un encoder visual, obtiene embeddings de imagen y los proyecta al espacio de un modelo de lenguaje. Luego el LLM recibe esos embeddings como contexto visual junto con una instrucción textual.

La secuencia conceptual es:

```

imagen -> encoder visual -> projector -> embeddings visuales compatibles con LLM
embeddings visuales + instruccion -> LLM -> respuesta

```

11.2. Código didáctico

```

class VisualProjector(nn.Module):
    """Proyector visual hacia el espacio de un LLM."""

    def __init__(self, visual_dim=768, llm_dim=4096):
        super().__init__()
        self.projector = nn.Sequential(
            nn.Linear(visual_dim, llm_dim),
            nn.GELU(),
            nn.Linear(llm_dim, llm_dim),
        )

    def forward(self, visual_embeddings):
        return self.projector(visual_embeddings)

```

11.3. Ejemplo manual

Entrada:

```

Imagen: gato con lentes sobre una mesa
Usuario: Describe solo lo visible.

```

Salida esperada:

```

Se observa un gato con lentes sobre una mesa.

```

Salida problemática:

```

El gato pertenece a una tienda de mascotas y esta esperando a su dueño.

```

La segunda salida es fluida, pero alucina información no visible. Por eso LLaVA y otros asistentes multimodales deben evaluarse por fidelidad visual, no solo por calidad lingüística.

12. Comparación técnica de variantes

Modelo	Entrada principal	Mecanismo central	Riesgo típico
BERT	tokens textuales	atención bidireccional encoder-only	confundir comprensión con generación
VisualBERT	texto y regiones visuales	secuencia conjunta texto-región	depender de regiones mal detectadas
UNITER	texto y regiones visuales	fusión profunda y alineamiento palabra-región	alto costo y sensibilidad al dataset
ViLT	texto y patches	Transformer sin detector externo pesado	menor estructura visual explícita
BLIP-2	imagen, consultas y texto	Q-Former entre visión congelada y LLM congelado	pérdida de detalles visuales en el cuello de botella
LLaVA	embeddings visuales e instrucción	proyección visual hacia LLM instruccional	alucinación y grounding débil

13. Ejemplos integradores

13.1. Ejemplo 1: ventana de contexto multimodal

Supongamos:

```
Tokens textuales: 12
Tokens visuales: 36
Tokens especiales: 2
Total: 50
```

La atención efectiva es de tamaño 50×50 por head y por capa. Si hay 12 heads y 12 capas, el modelo calcula muchas matrices de atención, pero cada una puede especializarse en relaciones distintas. Una head puede capturar objetos, otra atributos, otra relaciones espaciales y otra contexto global.

13.2. Ejemplo 2: clasificación con [CLS]

Supongamos:

```
h_[CLS] = [0.20, -0.10, 0.70, 0.05]
```

Cabecera lineal:

```
clase animal = [0.5, 0.1, 0.8, 0.0]
clase objeto = [0.1, 0.7, 0.2, 0.4]
```

Cálculo:

$$\begin{aligned} \text{score}(\text{animal}) &= 0.20(0.5) + (-0.10)(0.1) + 0.70(0.8) + 0.05(0.0) = 0.65, \\ \text{score}(\text{objeto}) &= 0.20(0.1) + (-0.10)(0.7) + 0.70(0.2) + 0.05(0.4) = 0.11. \end{aligned}$$

La clase predicha es **animal**. En modelos multimodales, la cabecera puede tomar [CLS] fusionado con evidencia visual.

13.3. Ejemplo 3: error de alineamiento

Imagen con tres regiones:

R1: persona
R2: bicicleta
R3: arbol

Texto generado:

Una persona monta una bicicleta junto a un arbol.

Evaluación manual:

Elemento	Evidencia esperada	Evidencia usada	Diagnóstico
persona	R1	R1	correcto
bicicleta	R2	R3	confusión de objeto
monta	R1 y R2	R1	relación incompleta
arbol	R3	R3	correcto

14. Evaluación y criterios de calidad

14.1. Qué debe medirse

Un sistema multimodal no debe evaluarse solo con una métrica. Para modelos derivados de BERT y arquitecturas multimodales, se recomienda medir:

- exactitud o VQA score para respuestas cerradas,
- Recall@K para recuperación imagen-texto,
- CLIPScore para alineamiento semántico aproximado,
- BLEU, METEOR, ROUGE, CIDEr o SPICE para captioning,
- análisis humano para fidelidad visual, alucinación y sesgo,
- robustez ante cambios de prompt, recorte, resolución o ruido.

14.2. Taxonomía mínima de errores

El análisis de errores debe clasificar, al menos:

- confusión de objeto,
- acción incorrecta,
- conteo incorrecto,
- relación espacial incorrecta,
- OCR incorrecto,
- atributo incorrecto,
- alucinación,
- sesgo,
- respuesta vaga.

15. Ruta conceptual final

La progresión completa es:

1. BERT enseña tokens, segmentos, posiciones, atención bidireccional, MLM y fine-tuning.
2. VisualBERT agrega regiones visuales a la secuencia textual.
3. UNITER refuerza el alineamiento fino entre palabras y regiones.
4. ViLT reemplaza regiones por patches y simplifica el pipeline visual.
5. BLIP-2 usa Q-Former para conectar visión congelada con un LLM congelado.
6. LLaVA proyecta embeddings visuales al espacio del LLM y permite instrucción visual.

La idea que une estos modelos es que el sistema debe convertir evidencia textual y visual en representaciones compatibles, fusionarlas mediante atención y producir una salida verificable. La calidad científica no depende solo de que el modelo responda bien en algunos ejemplos, sino de que el experimento sea reproducible, auditable, evaluado con métricas complementarias y acompañado por análisis de errores.

16. Glosario breve

Token	Unidad discreta o vectorial procesada por el modelo.
Embedding	Representación vectorial de un token, región, patch o consulta.
Segment embedding	Vector que indica a qué segmento pertenece un token.
Position embedding	Vector que codifica posición dentro de la secuencia.
Attention mask	Máscara que permite o bloquea atención entre posiciones.
MLM	Objetivo que predice tokens ocultos usando contexto bidireccional.
Head	Subespacio de atención dentro de multi-head attention.
Cabecera	Módulo final para clasificación, predicción de spans u otra tarea.
Grounding	Vínculo entre una salida lingüística y evidencia visual verificable.
Q-Former	Transformer con consultas aprendidas para extraer evidencia visual.
Proyector visual	Módulo que adapta embeddings visuales al espacio de un LLM.

17. Conclusión

BERT es más que un modelo textual histórico. Es una plantilla conceptual para entender cómo los Transformers procesan secuencias. Los modelos multimodales toman esa plantilla y la extienden: primero agregan regiones, luego patches, después consultas aprendidas y finalmente embeddings visuales compatibles con LLMs. Por eso entender BERT con detalle permite comprender el diseño, las fortalezas y los límites de VisualBERT, UNITER, ViLT, BLIP-2 y LLaVA.